

LearnSphere

A Learning Management System where students enroll, study course modules, submit assignments and track their progress, and instructors build courses and review work.

PROJECT	Learning Management System (LMS): LearnSphere
DEVELOPER	Abhyuday Saxena
INSTITUTION	Thapar Institute of Engineering and Technology
DOMAIN	Full Stack Development · Individual Major Project
FRONTEND	React 18, Vite 5, React Router 6, custom CSS
BACKEND	Node.js, Express 4, Firebase Admin SDK, Helmet
DATABASE	Supabase (PostgreSQL) with Row Level Security
AUTHENTICATION	Firebase Authentication: email/password and Google sign-in
HOSTING	Vercel (frontend) · Render (backend) · Supabase (database)

LINKS

Live application: <https://quillancemaajorproject1-client.vercel.app>

Source code: https://github.com/abhyudaysaxena06/QuillanceMaajorProject_1

Problem statement and objectives

Why the project exists and what it set out to do.

In many courses the material, the deadlines and the marks are spread across chat groups, shared drives and spreadsheets. Students lose track of what is due, and instructors cannot easily see who is falling behind or which work still needs marking.

LearnSphere brings these into one web application with separate, role-based workspaces for students, instructors and administrators. Course content, submissions and progress are stored in one database and served through a single REST API.

Objectives

- Secure registration and login, with role-based access for students, instructors and admins.
- Instructors can create, edit and delete courses, organise them into ordered modules and publish assignments.
- Students can browse the catalog, enroll, study module material and submit assignments.
- Progress is calculated from completed modules and shown as percentages, progress bars and status.
- Instructors review submissions, award marks, write feedback or request a resubmission.
- The project demonstrates the full stack: frontend, REST API, authentication, database and deployment.

Scope against the minimum completion criteria

REQUIREMENT FROM THE BRIEF	STATUS	WHERE
Login and registration	Done	Email/password and Google via Firebase
Student / Admin roles	Done	student, instructor, admin
Course management	Done	Instructor course editor
Course enrollment	Done	Course page, unique per student
Course modules and learning materials	Done	Ordered modules with notes, video and links
Assignment management and submission	Done	Text, GitHub, Drive or project URL
Progress tracking	Done	Per module, per course and overall
Student and admin dashboards	Done	Role-specific dashboards
Database integration	Done	Supabase PostgreSQL, 7 tables
Optional: search and filtering, password reset	Done	Catalog search, Firebase reset email

User roles and access control

Every rule below is checked on the server, not only hidden in the interface.

ROLE	CAPABILITIES	MAIN SCREENS
Student	Register and log in, browse and search courses, enroll or leave, study modules and mark them complete, submit and update assignments until graded, view marks and feedback, track progress.	Dashboard, Catalog, My Courses, Module view, Assignments, Profile
Instructor	Create, edit, publish and delete their own courses; add, reorder and delete modules with learning materials; create assignments with deadlines and maximum marks; review submissions; give marks and feedback or request a resubmission; view the roster with each student's progress.	Dashboard, Manage Courses, Course Editor, Submissions
Admin	Everything an instructor can do, on every course. Also: view all users, change roles, activate or deactivate accounts, and see platform statistics.	Dashboard, Users, all instructor screens

How a request is authorised

1. The user signs in with Firebase (email/password or Google). Firebase stores the password as a salted hash and returns a short-lived signed ID token (a JWT).
2. The React client sends that token with every API call in the `Authorization: Bearer` header.
3. The `authenticate` middleware verifies the token with the Firebase Admin SDK, then loads the matching row from the `users` table, creating it on first login. Deactivated accounts are rejected with 403.
4. `requireRole()` checks the role for the route, and ownership helpers (`assertCanManage`, `isEnrolled`) check that the user owns the course or is enrolled in it.

The browser never talks to the database. Row Level Security is enabled on every table with no public policies, so the public Supabase key can read nothing. Only the server, holding the service-role key, can query data.

Onboarding

On first login a new user chooses *I want to learn* or *I want to teach*. Emails listed in the server's `ADMIN_EMAILS` setting are made admins automatically. Admins can change any user's role later.

Functional modules

Authentication

- Registration with name, email and password, with form validation
- Clear errors for a wrong password or an already-registered email
- Google sign-in and password reset by email
- Protected routes and logout

Student dashboard

- Enrolled and completed courses, modules done, pending work, average marks
- Per-course progress bars
- Upcoming deadlines
- Activity log of finished modules, submissions and grades

Course management

- Create, edit, delete; draft or published
- Title, description, category, difficulty, duration, image
- Catalog with search and category filter

Enrollment

- One-click enroll and leave
- Duplicate enrollment blocked by the API and a unique constraint
- Enrollment date, progress percentage and status

Modules and materials

- Ordered modules, reordered with up/down controls
- Notes, embedded YouTube/Vimeo video, duration
- Links to notes, PDFs, videos, source code, references and exercises

Assignments

- Instructions, deadline, maximum marks
- Submit text and/or a GitHub, Drive or project link
- Update until graded; late submissions flagged

Review and marks

- Central Submissions page: awaiting review, graded, resubmission requested
- Marks validated against the maximum, plus written feedback
- Status: submitted, graded or resubmit

Progress and admin

- Segmented progress bar, one segment per module
- Instructor roster with each student's progress
- Admin user list: role changes, activation, enrollment counts

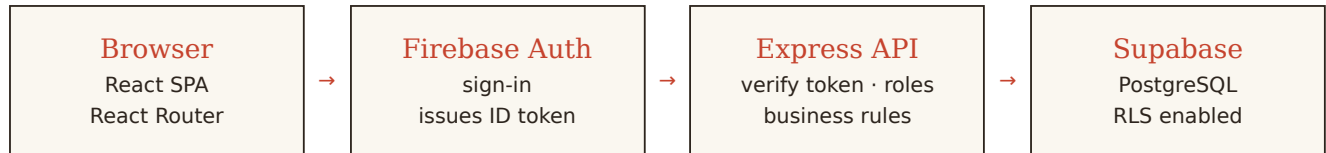
User workflows

Student: Register → Login → Browse courses → Enroll → Open course → Study modules → Mark complete → View assignments → Submit → Track progress → Course completed.

Instructor: Login → Dashboard → Create course → Add modules → Publish assignment → Monitor students → Review submissions → Give marks and feedback → Track progress.

System architecture

A React single-page app talks to an Express REST API; the API is the only component that reaches the database.



Project structure

client/	React frontend (Vite)
src/api.js	fetch wrapper, attaches the Firebase ID token
src/context/	AuthContext: session, profile, login/register/logout
src/components/	Layout, ProtectedRoute, CourseCard, ProgressBar, GradeForm
src/pages/	Home, Login, Onboarding, Dashboard, Courses, CourseDetail, LessonView, MyLearning, Assignments, AssignmentDetail, Teach, CourseEditor, Submissions, Admin, Profile
server/	Express backend
src/config/	Firebase Admin and Supabase clients
src/middleware/	authenticate, requireRole
src/routes/	auth, courses, lessons, assignments, dashboard, admin
scripts/seed-demo.js	creates demo accounts and sample progress
database/	schema.sql, seed.sql, setup_all.sql, ER diagram

Design decisions

- **Firestore for identity, Supabase for data.** Firestore handles password hashing, Google OAuth and password reset; Supabase gives a managed relational database for the course data.
- **Progress is calculated, not stored.** Course progress is completed modules divided by total modules, computed from `lesson_progress` on each request, so it cannot go out of sync when modules are added or removed.
- **Input whitelisting.** Each route copies only allowed fields from the request body before writing to the database.
- **Security headers and CORS.** Helmet sets HTTP security headers; CORS allows only the configured frontend origins.

Interface

The interface uses a warm paper-and-ink theme (cream background, dark brown text, one terracotta accent) with serif headings, a numbered top navigation that becomes a full-screen menu on phones, and a few subtle transitions. It is responsive from 360 px phones to desktop.

Database schema

Seven PostgreSQL tables in Supabase. UUID primary keys, foreign keys with cascading deletes.

TABLE	KEY FIELDS	CONSTRAINTS
users	firebase_uid, email, name, role, onboarded, is_active, bio	firebase_uid unique; role in student / instructor / admin
courses	title, description, category, level, duration, thumbnail_url, published, instructor_id	instructor_id → users; level check
lessons (modules)	course_id, title, content, video_url, position, duration_minutes, resources (JSON)	course_id → courses; indexed by (course_id, position)
enrollments	user_id, course_id, enrolled_at	unique (user_id, course_id)
lesson_progress	user_id, lesson_id, completed_at	unique (user_id, lesson_id)
assignments	course_id, title, description, due_date, max_points	max_points > 0
submissions	assignment_id, student_id, content, link_url, status, grade, feedback, graded_at	unique (assignment_id, student_id); status check

REST API (selected endpoints)

METHOD	ENDPOINT	PURPOSE	ACCESS
GET	/api/public/courses	Featured courses for the home page	Public
GET	/api/auth/me	Current profile (created on first login)	Signed in
POST	/api/auth/onboard	Choose student or instructor	Signed in
GET	/api/courses	Catalog with search and category	Signed in
POST	/api/courses	Create course	Instructor
POST	/api/courses/:id/enroll	Enroll in a course	Student
POST	/api/lessons	Add a module	Owner
POST	/api/lessons/:id/complete	Mark module complete	Enrolled
POST	/api/assignments/:id/submit	Submit or resubmit work	Enrolled
PATCH	/api/assignments/submissions/:id/grade	Marks, feedback, status	Owner
GET	/api/dashboard	Role-specific statistics	Signed in
PATCH	/api/admin/users/:id	Change role, activate or deactivate	Admin

The full list of 25 endpoints is in docs/API.md. Errors are returned as { "error": "message" } with status 400, 401, 403, 404, 409 or 500.

Setup, deployment and demo accounts

Local setup

```
# 1. clone and install
git clone https://github.com/abhyudaysaksena06/QuillanceMaajorProject_1.git
cd QuillanceMaajorProject_1
npm run setup

# 2. database: run database/setup_all.sql in the Supabase SQL Editor
# 3. fill in server/.env and client/.env (see the .env.example files)

# 4. demo accounts and sample progress
npm run seed:demo

# 5. start API (port 5000) and web app (port 5173) together
npm run dev
```

Production deployment

- **Frontend** on Vercel (root client, Vite preset). vercel.json rewrites every path to index.html so client-side routes survive a refresh.
- **Backend** on Render (root server, npm start). Secrets are stored as Render environment variables.
- **Database** on Supabase; **authentication** on Firebase, with the Vercel domain added to Authorized domains.

Demo accounts

ROLE	EMAIL	PASSWORD	PREPARED DATA
Student	student@learnsphere.demo	Demo@1234	2 courses, modules done, a graded assignment
Student	student2@learnsphere.demo	Demo@1234	1 course, a submission awaiting review
Instructor	instructor@learnsphere.demo	Demo@1234	Owns the 3 sample courses
Admin	admin@learnsphere.demo	Demo@1234	User management and statistics

Testing

The testing checklist from the brief was covered manually (docs/TESTING.md): valid and invalid login, duplicate registration, students blocked from instructor and admin pages and APIs (403), course and assignment CRUD, duplicate enrollment rejected (409), submissions stored, progress updated after module completion, form validation, mobile layout, and data persisting after a restart.

Outcome and future work

LearnSphere is a working full-stack LMS: administrators and instructors manage courses, modules and assignments; students enroll, complete modules, submit work and follow their progress. The project shows a React frontend, an Express REST API, Firebase authentication with role-based authorization, a relational database with enforced constraints, and cloud deployment working together.

Challenges

- Combining two services for auth and data: solved by verifying Firebase tokens on the server and mapping each Firebase user to a database row.
- Keeping data private while using a hosted database: solved with Row Level Security and a server-only service key.
- Accurate progress when course content changes: solved by computing progress from completed modules instead of storing a number.
- Cross-origin requests between Vercel and Render: solved with a configurable CORS allow-list.

Future improvements

- Quizzes at the end of each module
- Certificates when a course is completed
- Email notifications for new assignments and marks
- File uploads for submissions using Supabase Storage
- Discussion forum per course, course ratings and analytics charts

Submission contents

ITEM	LOCATION
Frontend and backend source	client/, server/
Database schema and sample data	database/
README and project documentation	README.md, docs/, this PDF
Output screenshots	Screenshots/
1-minute demonstration video	Demo Video.mp4
Deployment link	Cover page and README